Guide du style C

GUIDE PRATIQUE · VERSION 0.29-DEV

Écrire un code lisible, comprendre les diagnostics et préparer ses exercices de la Piscine C.

Ce guide accompagne Moulinette Lab. Il présente les règles les plus utiles du profil 42 autodidacte, des exemples et les erreurs fréquentes. Il ne remplace ni les sujets, ni la Norme officielle, ni les consignes d'un campus.

Important : les tests sont non officiels et ne prouvent pas une conformité complète. La Norminette contrôle le style ; le compilateur contrôle notamment la validité du C ; les tests fonctionnels contrôlent certains comportements. Ce sont trois choses différentes.

01 · Le profil du Lab

Quelle norme ?

Moulinette Lab utilise la Norminette **3.3.59**, outil externe, pour les fichiers C et en-têtes concernés. Pour le profil autodidacte, **seul le diagnostic** `INVALID_HEADER` **est ignoré** : l'en-tête institutionnel 42 n'est pas exigé par le Lab. Les autres diagnostics restent pris en compte. Si la Norminette ou la version attendue est indisponible, le style n'est pas validé.

Le document de référence de la version 3.3.59 décrit la Norme version 4. Les règles et outils peuvent changer ; un sujet ou campus peut imposer des exigences supplémentaires. Ce guide résume les règles utiles et ne prétend pas être exhaustif.

Lire une correction

- **Compilation** : le programme peut-il être compilé avec les options du Lab ?
- **Prototype et liaison** : le nom, les types et la liaison attendus sont-ils vérifiés pour l'exercice ? Tous les exercices ne contrôlent pas chaque aspect du prototype.
- **Tests fonctionnels** : la sortie ou les valeurs correspondent-elles aux cas exécutés ?
- **Norminette adaptée** : les diagnostics de style, autres que l'en-tête ignoré, sont-ils acceptés ?

Si une étape échoue, les suivantes peuvent ne pas être exécutées. Ne confondez pas « non testé » et « réussi ».

02 · Présentation et noms

Indentation et lignes

- Indentez avec de **vraies tabulations**, affichées à la largeur de quatre espaces ; quatre espaces ne remplacent pas une tabulation.
- Limitez une ligne à **80 colonnes**, commentaires compris ; une tabulation occupe la largeur jusqu'au prochain alignement.
- Une fonction fait au maximum **25 lignes**, sans compter ses accolades extérieures.
- Une instruction par ligne. Pas d'espaces en fin de ligne ; une ligne vide ne contient ni espaces ni tabulations.
- Placez les accolades et structures de contrôle sur les lignes attendues ; séparez deux fonctions par une ligne vide.

Réglez votre éditeur pour montrer les caractères invisibles et insérer des tabulations. Le rendu d'un PDF ne permet pas de distinguer de façon fiable une tabulation d'espaces : utilisez toujours votre éditeur pour vérifier les fichiers.

Noms

Pour les variables et fonctions, utilisez des noms explicites en anglais, en minuscules et en `snake_case`, par exemple `current_char` et `ft_print_alphabet`. Les structures, types, unions et énumérations ont des préfixes spécifiques (`s_`, `t_`, `u_`, `e_`). Vérifiez aussi le nom exact demandé par l'exercice : un nom lisible mais différent ne satisfait pas le prototype attendu.

03 · Fonctions et variables

À respecter

- Une fonction sans paramètre est déclarée avec `(void)` ; une fonction peut prendre au maximum quatre paramètres nommés.
- Placez les déclarations de variables au début du bloc et séparez-les des instructions par une ligne vide. Le document de référence indique cinq variables au maximum par bloc.
- Déclarez une variable par ligne ; ne l'initialisez pas sur la même ligne, sauf exceptions de la Norme pour certaines variables `static` ou globales autorisées.
- Placez l'astérisque d'un pointeur près de son nom : `char *text`. Alignez les déclarations avec des tabulations selon le contexte.
- Une fonction qui retourne une valeur doit retourner une valeur du type approprié sur les chemins nécessaires ; une fonction `void` ne retourne pas de valeur. La Norme demande des parenthèses autour de l'expression retournée.

Exemple simple

```
int ft_next(int value)
{
    int result;

    result = value + 1;
    return (result);
}
```

Dans cet exemple, les retraits et l'alignement sont des tabulations dans le fichier source. Si vous recopiez depuis le PDF, vérifiez-les dans l'éditeur.

04 · Structures et en-têtes

Structures de contrôle

Dans le profil de la Norme étudiée, utilisez notamment `while` pour vos boucles. `for`, `do ... while`, `switch` et l'opérateur ternaire sont interdits. N'utilisez pas les variables globales sauf si le sujet les autorise ou les impose. Des règles supplémentaires s'appliquent à d'autres constructions : consultez la Norme et le sujet.

Les espaces et parenthèses comptent : par exemple `while (c <= 'z')` et `c = c + 1;`. Utilisez une expression et une instruction claires par ligne.

Fichiers .h

Un en-tête contient des déclarations, prototypes, inclusions et définitions autorisés. Protégez-le contre les inclusions multiples avec `#ifndef`, `# define` et `#endif`. Les inclusions doivent être justifiées ; n'incluez pas un fichier `.c` dans votre rendu. Respectez les noms et structures imposés par le sujet.

```
#ifndef FT_EXAMPLE_H
# define FT_EXAMPLE_H

int ft_next(int value);

#endif
```

Cet exemple illustre une protection d'en-tête ; chaque sujet peut demander des noms ou règles supplémentaires. Le Lab peut fournir lui-même certains en-têtes de test : référez-vous au sujet et au rapport de l'exercice.

Complément · Fichiers et commentaires

Organisation des fichiers

Un fichier `.c` ne doit pas contenir plus de **cinq définitions de fonctions** dans la Norme de référence. Les inclusions se placent en début de fichier et doivent être utiles. Ne rendez que les fichiers demandés par le sujet. Pour un exercice de fonction, votre `main` personnel reste dans un fichier de test séparé ; pour un programme, suivez les exigences de l'exercice.

Commentaires et préprocesseur

Les commentaires doivent être utiles et en anglais ; le document de référence ne les autorise pas à l'intérieur des fonctions. Les noms de macros sont en majuscules. Les `#define` servent aux valeurs littérales et constantes, et non à contourner les règles ; les macros multilignes sont interdites. Les directives et protections d'en-tête ont leur indentation particulière.

Constructions à éviter

La Norme citée interdit également `goto`, les tableaux à taille variable (VLA) et les types implicites dans les déclarations. Évitez l'affectation multiple. Certaines règles demandent une appréciation humaine : le résultat de la Norminette ne remplace pas une relecture.

Makefile

La Norminette ne vérifie pas les Makefiles. Selon le sujet et la Norme de référence, les règles `all`, `clean`, `fclean`, `re` et la cible du programme peuvent être exigées ; une reconstruction inutile lors d'un second `make` est à éviter. Les contrôles du Lab sont partiels : lisez les consignes de l'exercice.

05 · Exemple : alphabet

Pour un exercice demandant `ft_print_alphabet`, une fonction de référence pédagogique peut ressembler à ceci :

```
#include <unistd.h>

void    ft_print_alphabet(void)
{
    char    c;

    c = 'a';
    while (c <= 'z')
    {
        write(1, &c, 1);
        c++;
    }
}
```

Le fichier source de cet exemple a été vérifié avec Norminette 3.3.59 : seul `INVALID_HEADER` a été signalé puis ignoré par l'adaptateur du Lab. Cela ne garantit pas que tout copier-coller, ni tout autre exercice, passera les vérifications.

La fonction n'a pas de `main` : pour cet exercice de fonction, le correcteur fournit son programme de test. Pour un exercice qui demande un programme, `main` peut au contraire être nécessaire. Suivez toujours le sujet.

Le fichier à rendre pour C00/ex01 s'appelle `ft_print_alphabet.c`, placé seul dans son dossier de rendu pour le Lab actuel. Ne supprimez pas vos fichiers personnels : faites une copie dans un dossier de test séparé.

06 · Erreurs fréquentes

Compilation

- `unused variable` : une variable est déclarée mais non utilisée. Supprimez-la si elle n'est pas utile ou utilisez-la si elle est nécessaire. Le Lab compile avec `-Wall -Wextra -Werror` : des avertissements deviennent des erreurs.
- `control reaches end of non-void function` : une fonction censée renvoyer une valeur peut finir sans `return`. Vérifiez le type et les chemins de retour ; ne changez pas la signature si le sujet impose un type précis.
- `undefined reference` : le nom attendu ou une définition manque à la liaison. Vérifiez orthographe, signature et fichiers du rendu. Une erreur de liaison ne prouve pas à elle seule laquelle de ces causes est présente.
- `multiple definition of main` : plusieurs programmes principaux sont liés. Pour un exercice de fonction, gardez votre `main` d'essai hors du rendu ; pour un exercice de programme, vérifiez les règles du sujet.

Style et comportement

Une erreur de tabulation ou de longueur de ligne concerne la présentation ; une sortie incorrecte concerne le comportement. Corrigez la première erreur claire, relancez, puis lisez la suite : toutes les étapes peuvent ne pas avoir été exécutées. Un résultat « incomplet » n'est pas une réussite.

07 · Méthode et ressources

Avant de lancer la correction

- Relisez les fichiers, prototypes, fonctions autorisées et comportement demandés par le sujet.
- Gardez une copie de vos rendus ; placez les fichiers exigés dans le dossier de l'exercice, sans fichiers supplémentaires lorsqu'ils sont interdits.
- Vérifiez l'indentation, la taille des fonctions, les déclarations et les noms.
- Lancez Moulinette Lab sur votre propre code de confiance, puis ouvrez les détails du rapport et le tableau de bord.
- Traitez d'abord la compilation, puis le comportement et le style ; ne considérez pas les étapes non exécutées comme réussies.

Sources et limites

Norme française version 4 du dépôt officiel, associée au tag Norminette 3.3.59 :

https://raw.githubusercontent.com/42School/norminette/3.3.59/pdf/fr.norm.tex

Dépôt de l'outil : https://github.com/42School/norminette

Le Lab n'est pas affilié à 42, n'utilise pas les tests officiels et ignore seulement `INVALID_HEADER` dans ce profil. Les consignes du sujet et celles de votre campus priment pour une évaluation officielle. Les règles et diagnostics peuvent évoluer selon les versions.